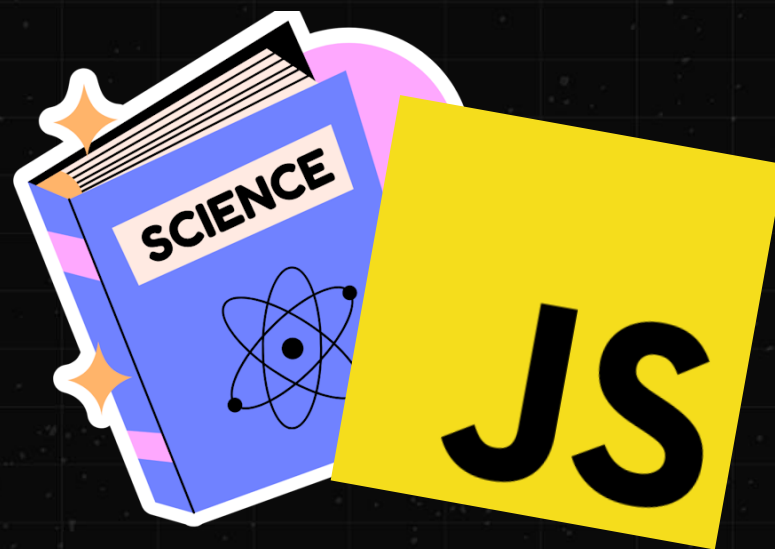
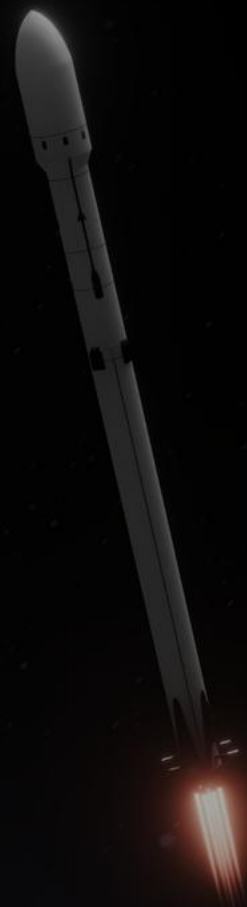


ЛЕКЦИЯ #1

CS BO FRONTEND



ПРО ЧТО
ДАННЫЙ
КУРС?



БАЗА ПРОГРАММИРОВАНИЯ

- ✱ Мы начинаем путешествие длиною в 6 месяцев, чтобы **укрепить базу**
- ✱ И начать применять её для **программирования на JS и для задач Frontend**
- ✱ Но полученные **знания будут универсальны** и применимы где угодно

КУРС ПОЗВОЛИТ ИДТИ ДАЛЬШЕ

- * Подготовит к изучению WebAssembly, WebGPU и других низкоуровневых возможностей
- * Писать изящный и эффективный код
- * Не бояться ИИ, а использовать его для повышения своей эффективности

ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Поговорить про само понятие вычисления
- * Кодирование информации
- * Системы счисления
- * И типы данных

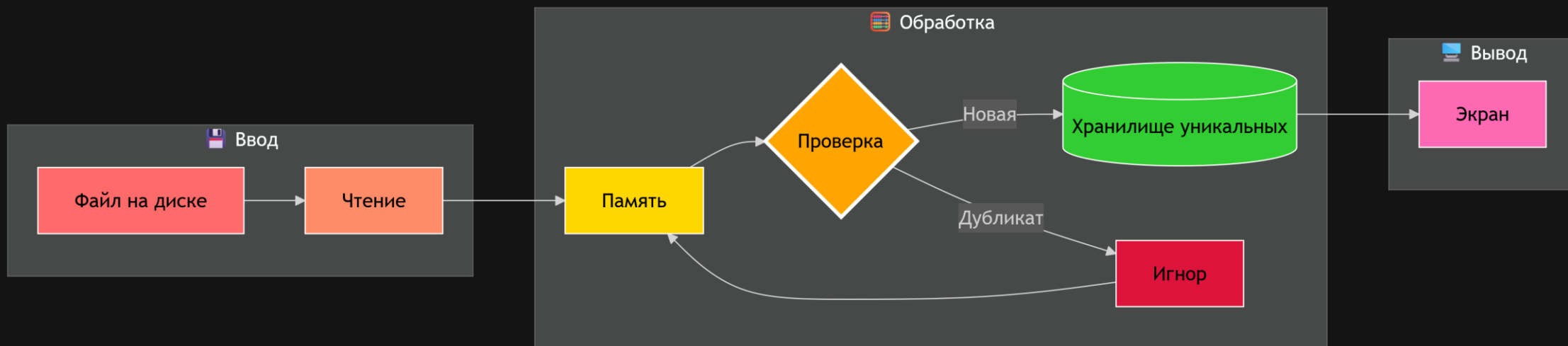


ЧТО ТАКОЕ ПРОГРАММИРОВАНИЕ?

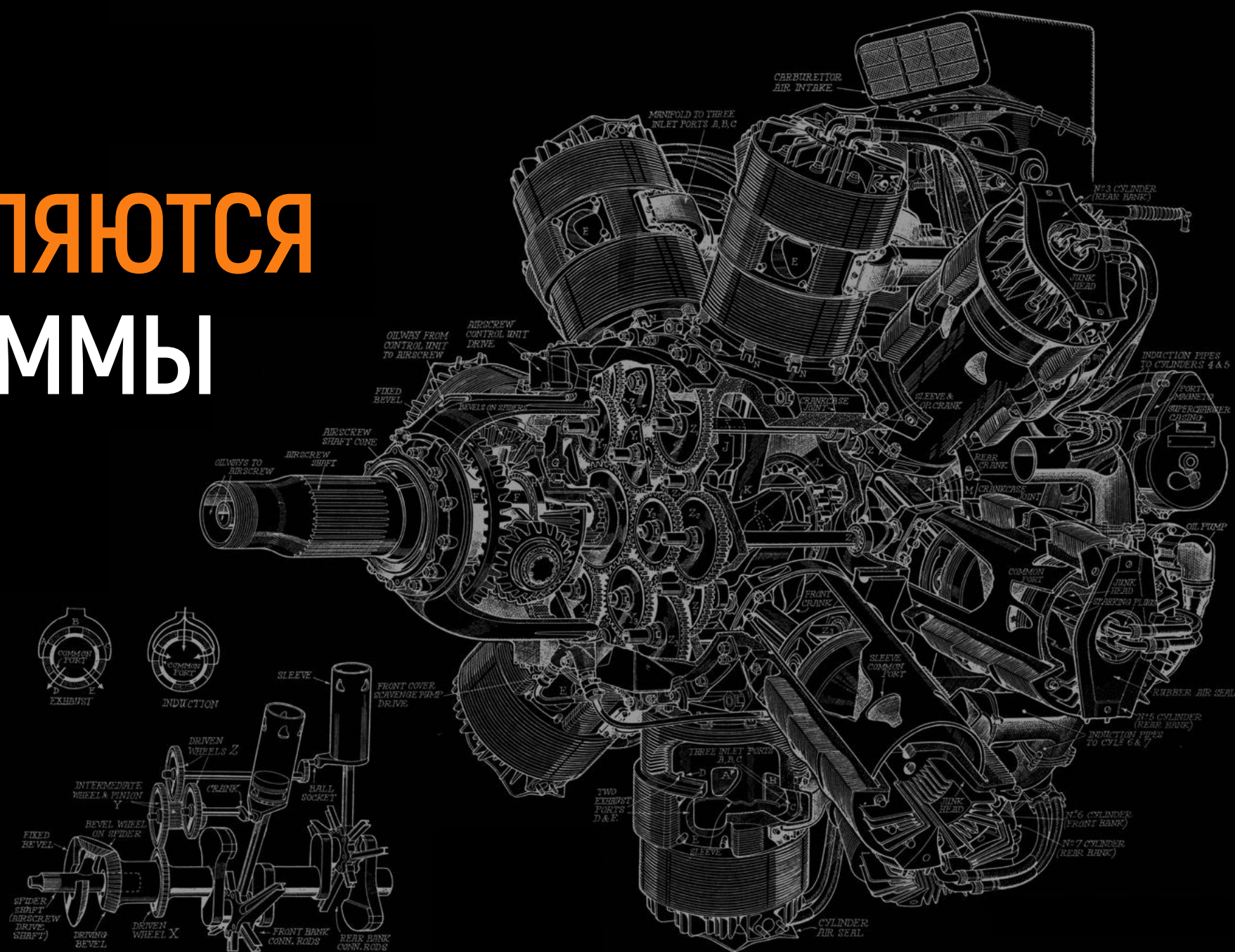
ПРОГРАММИРОВАНИЕ

- * Это **вычисления над данными**
- * У нас есть некоторое устройство умеющее делать какие то вычисления
- * Назовем его вычислитель или процессор
- * Нужно **представить данные в понятный для него формат**
- * Используя набор предлагаемых инструкций осуществить вычисление
- * Вывести результат

НАПРИМЕР



КАК ВЫЧИСЛЯЮТСЯ ПРОГРАММЫ



ВСЕ НАЧИНАЕТСЯ С ВЫЧИСЛИТЕЛЯ

- * Это устройство или программа представляющее «виртуальное» устройство
- * Вычислитель **поддерживает фиксированный набор операций**
- * Тогда любая программа – это просто **последовательность операций**
- * А её **вычисление** – это чтение и выполнение операций в указанном порядке

СЕЙЧАС МИРОМ ПРАВЯТ ПОЛУПРОВОДНИКИ

- * Транзисторные компьютеры составляют основную часть всех устройств
- * Но **есть множество и других подходов и технологий**
- * Релейные и ламповые
- * Квантовые и фотонные
- * И даже механические
- * Простейший пример – **обычные счеты**

РАЗНОСТНАЯ МАШИНА ЧАРЛЬЗА БЭББИДЖА



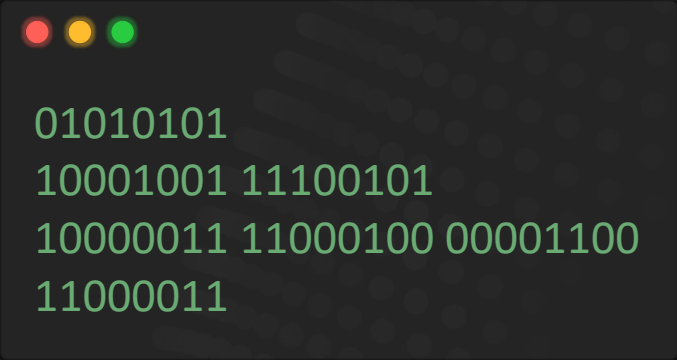
А КАК **НАПИСАТЬ** ПРОГРАММУ ДЛЯ ВЫЧИСЛИТЕЛЯ?



А ВОТ НАЧИНАЕМ ИНТЕРЕСНОЕ

- * «Язык» устройства тесно связан с его физической моделью
- * Электронные устройства используют бинарный язык – **«ток есть» и «тока нет»**
- * С математической точки зрения это **можно представить как 0 и 1**
- * Значит любая программа – это просто последовательность «этих цифр»
- * Часть из них – **отвечает за операции**
- * Часть – **за данные**

НАПРИМЕР

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It contains four lines of green binary code.

```
01010101  
10001001 11100101  
10000011 11000100 00001100  
11000011
```

РЕФЛЕКСИРУЕМ

- * Наша программа представляет собой поток двоичных цифр – бит
- * Одним битом много не закодируешь (всего 2 значения), но уже двумя битами – четыре, тремя – 8 и так далее
- * Например, **00** или **01** или **10** или **11**
- * Минимальная группа бит обрабатываемая процессором называется байт
- * Обычный размер байта составляет 8 бит

НЕ ТОЛЬКО БИТЫ И БАЙТЫ

- * Каждое вычислительное устройство **имеет набор поддерживаемых операций**
- * У каждой операции есть свой код (опкод)
- * У разных вычислителей могут быть **разные опкоды** даже для одних и тех же операций
- * Программа написанная под одно устройство **скорее всего не заработает на другом**
- * Чтобы улучшить переносимость придумали стандартные архитектуры, например x86 или ARM

ПОЧЕМУ БАЙТ
СОСТОИТ ИЗ 8
БИТ?



ЭТО НЕ ВСЕГДА ТАК

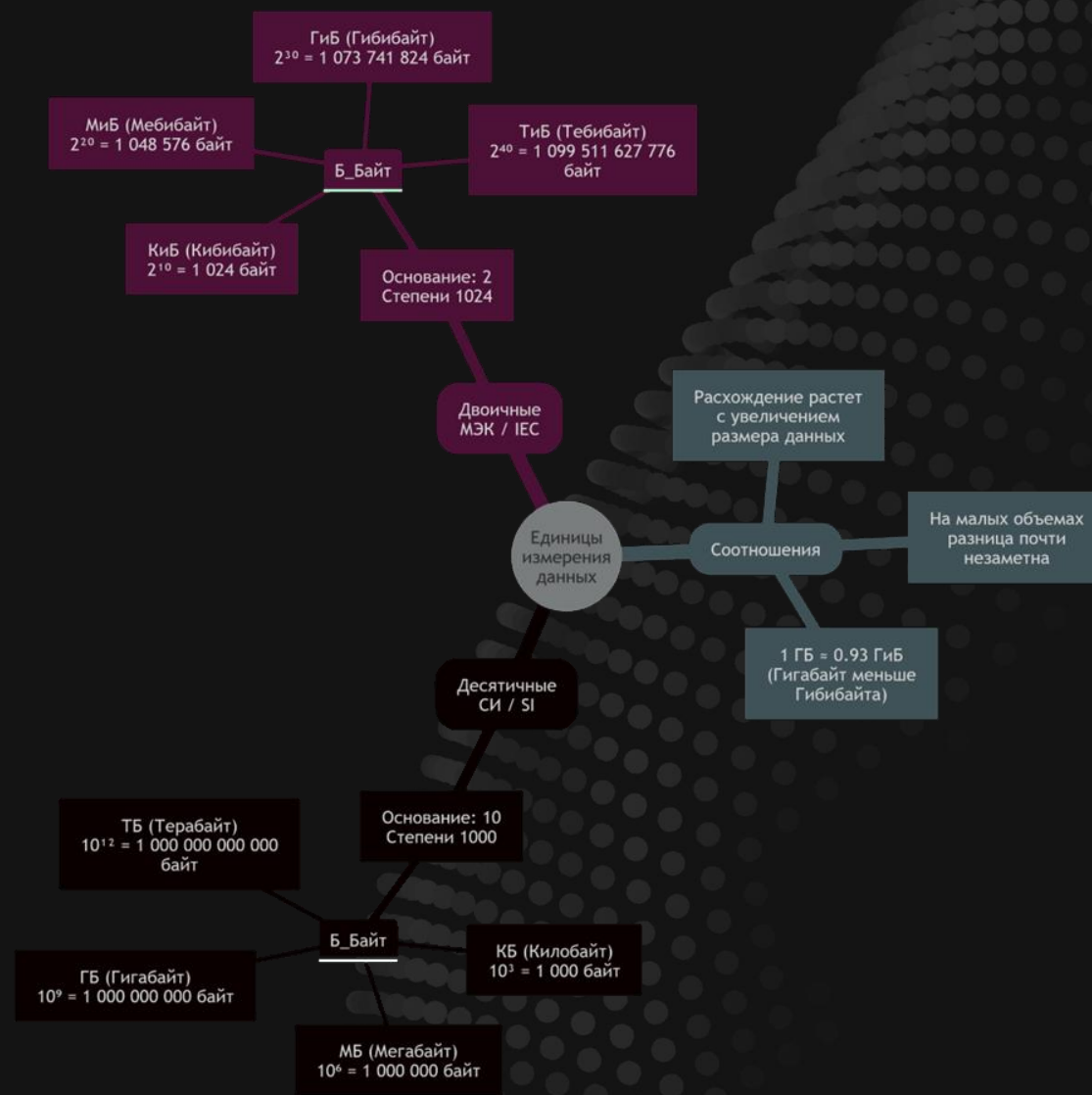
- * Есть вычислители, где **размер байта может отличаться**
- * Размер байта определяется таким образом, чтобы **«вместить» в себе наиболее часто используемый тип информации**
- * В большинстве «обычных» компьютеров **размер байта равен 8-ми битам**
- * Это обусловлено историческими причинами
- * И удобством для представления текстовой информации

КРАТНОСТЬ ДВУМ

- * Легко заметить, что **«размеры» информации в компьютере кратны двум** (делятся без остатка на два)
- * Причины этого кроются в том, что с одной стороны **один бит может содержать в себе 2 значения (0 и 1)**
- * С другой стороны, вычислитель работает с байтами, а не отдельными битами

КРАТНОСТЬ РАЗМЕРУ БАЙТА

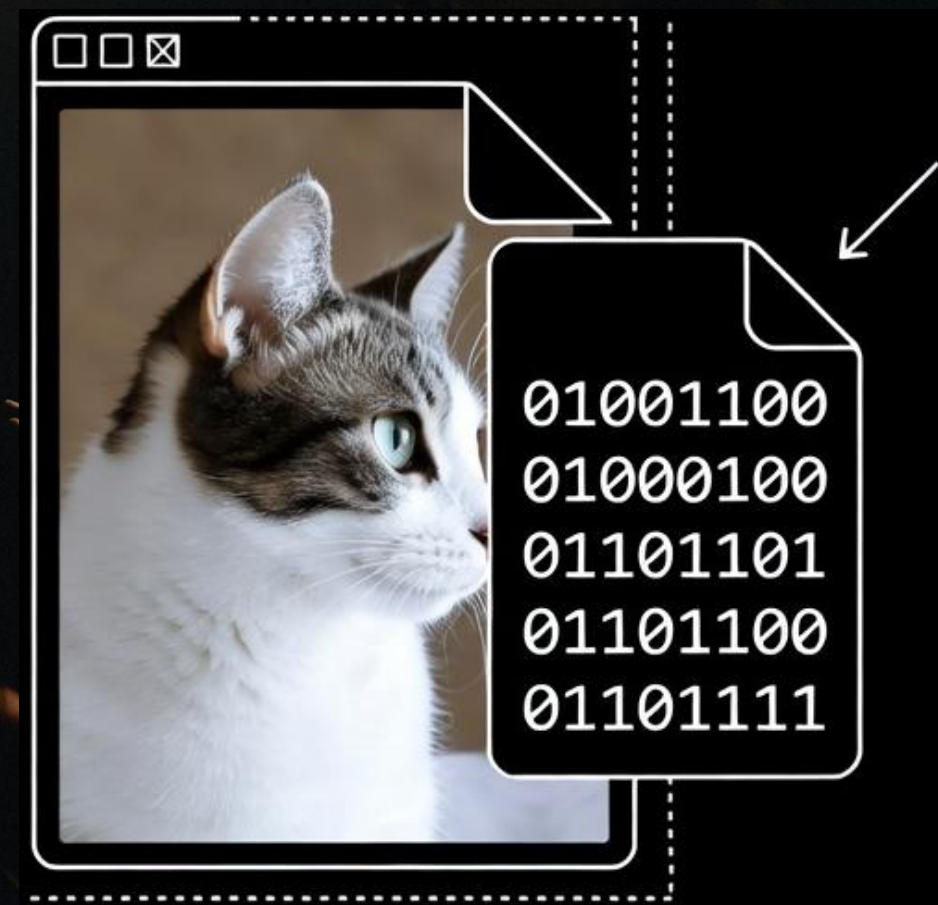
- * Байт – это **минимальный объём информации**, с которым может работать процессор
- * Любая представляемая информация в компьютере исчисляется в байтах
- * Существуют стандартные единицы измерения основанные на кратности байту и биту
- * Киббайт – 1024 байт (не кило)
- * В первую очередь это относится к оперативной памяти компьютера
- * Но если скажите килобайт, то **вас все равно поймут** 😊



КАК ВЫЧИСЛИТЕЛЬ «ВЫЧИСЛЯЕТ»

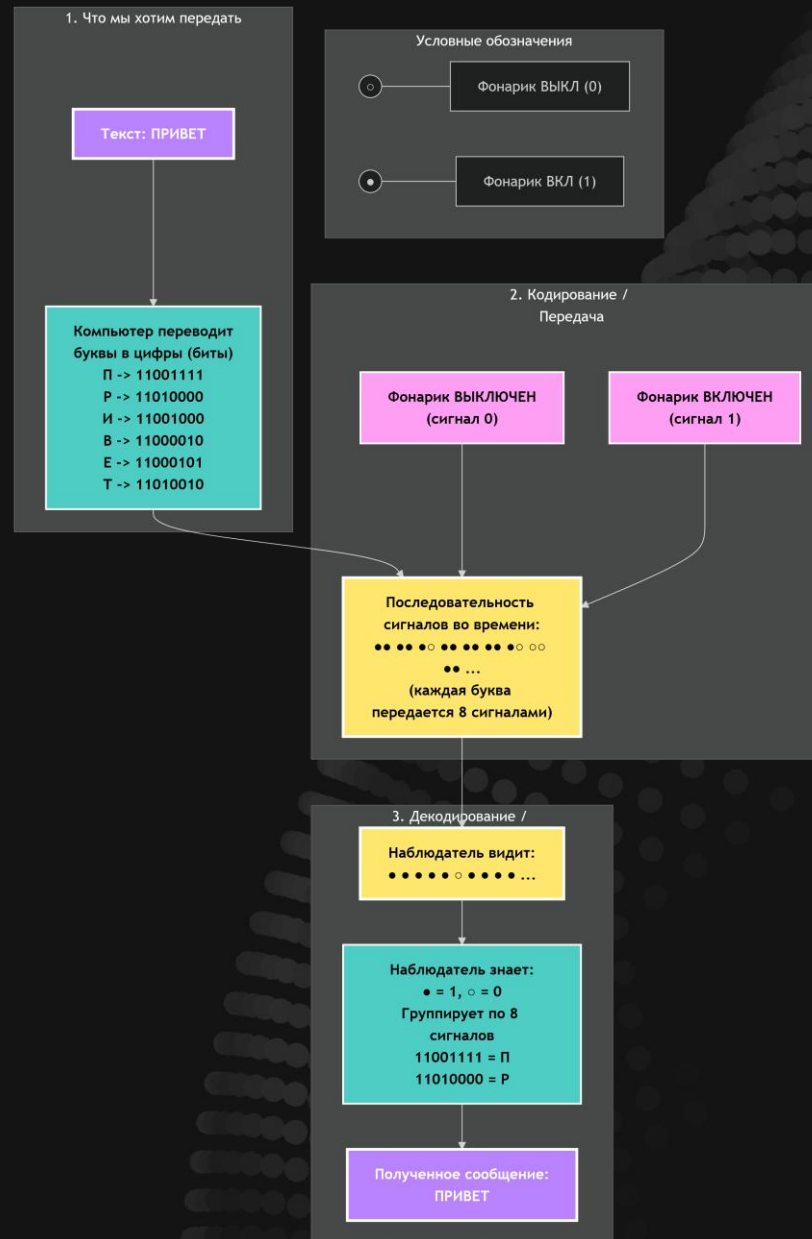
- * Мы не будем на курсе разбирать физическое устройство компьютера
- * Но, **любая инструкция процессора состоит из множества «логических вентилей»** по типу «и», «или», «или-не» и т.д.
- * Если вам интересна эта тема, то рекомендую для начала [«Код. Тайный язык информатики»](#)
- * [Магия транзисторов](#)
- * [Физика компьютерной памяти](#)

НО КАК ПРЕДСТАВИТЬ ИНФОРМАЦИЮ?



НАМ НУЖНО ЕЁ ЗАКОДИРОВАТЬ

- * Кодирование – это **процесс преобразования информации** из одной формы в другую
- * Декодирование – это процесс **преобразования информации в «исходную» форму**
- * Устная или письменная речь тоже является кодированием
- * Как и рисование, запись музыки в виде нот и т.д.
- * QR-коды, штрихкоды – это тоже примеры кодирования информации



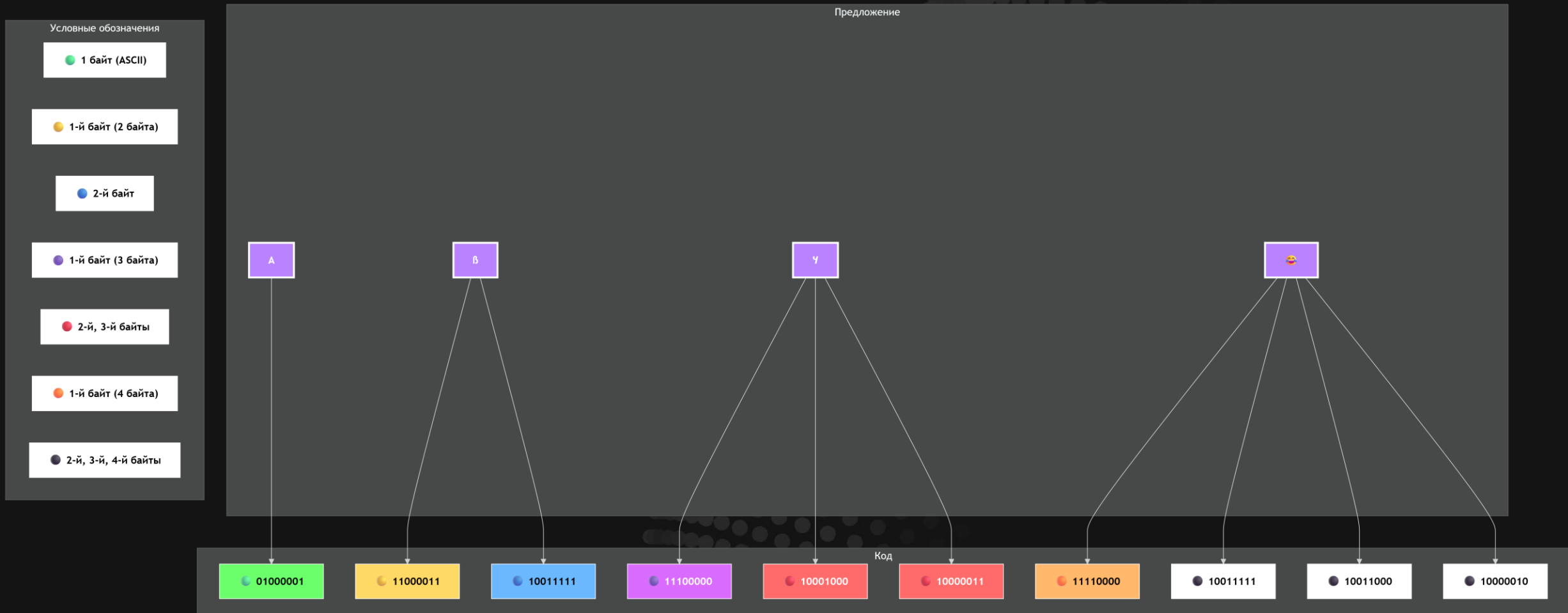
МОЖНО ЛИ
КОДИРОВАТЬ БУКВЫ
РАЗНЫМ КОЛИЧЕСТВОМ
БИТ?



МОЖНО

- * Это позволит передавать наиболее частые буквы **меньшим количеством бит**
- * Такой подход **часто применяется в алгоритмах сжатия информации**
- * Но в таком случае **усложняется процесс декодирования**
- * Если размер наших данных имеет фиксированный размер, то нам **проще и эффективнее работать с такими данными**
- * **У нас два стула:** меньше памяти или быстрее работать

НАПРИМЕР, UTF-8



У РАЗНЫХ ДАННЫХ
РАЗНОЕ ПРЕДСТАВЛЕНИЕ?



ЗАЧАСТУЮ

- * **Разные типы чисел:** целые, дробные, комплексные и так далее
- * **Разные виды строк** для текста
- * Разные форматы изображений, видео и звука
- * У каждого формата свой **алгоритм кодирования и декодирования**
- * Свои плюсы и минусы

НА ПРИМЕРЕ ЧИСЕЛ

ЯЗЫК МАТЕМАТИКИ СТАНДАРТИЗИРОВАН

- * Все используют индо-арабские цифры
- * И десятичную позиционную систему счисления
- * Вычисление арифметических выражений – это одна из важнейших задач компьютера
- * Процессоры «нативно» поддерживают разные виды чисел и операции над ними

НО КАК **ЗАПИСАТЬ**
ЧИСЛО В КОМПЬЮТЕР?

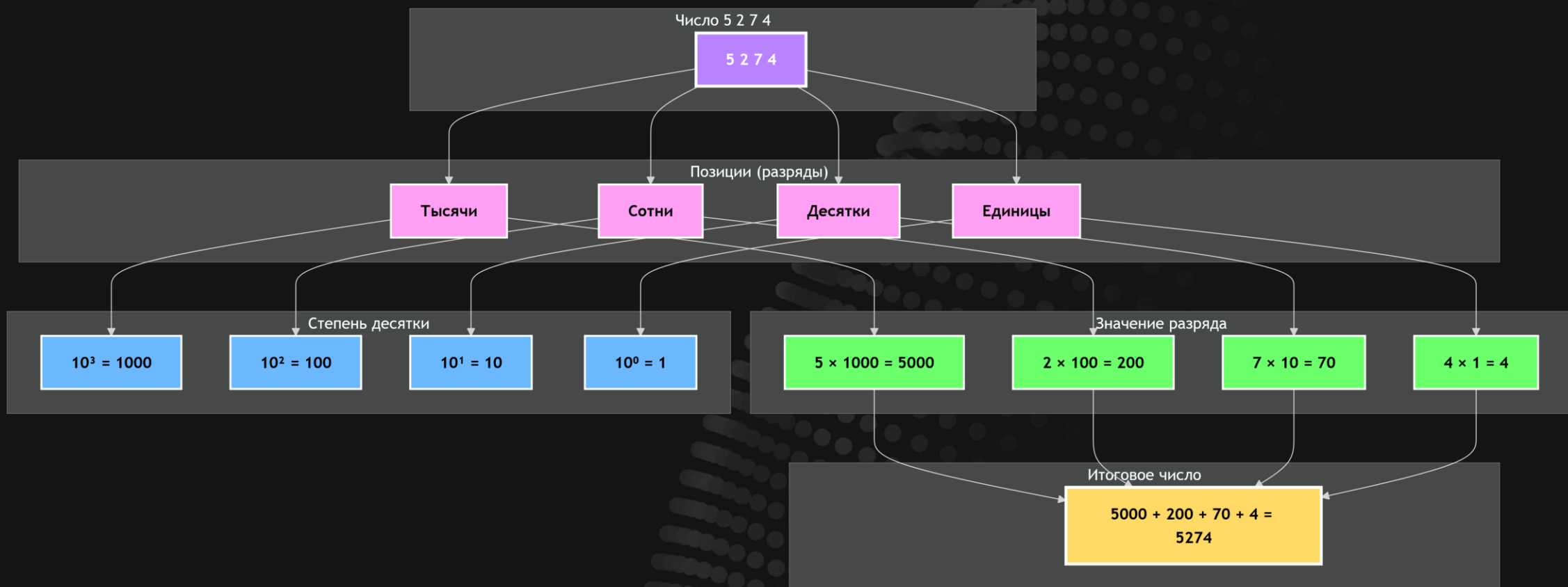


СИСТЕМЫ СЧИСЛЕНИЯ

- * Описывают правила **записывания и чтения чисел**
- * **Человек привык** думать в десятичной системе (10 пальцев)
- * Это позиционная система
- * Например, 100500 – в зависимости **от позиции цифры в числе меняется смысл**
- * Компьютер, как правило, понимает **только двоичную систему счисления**
- * **Надо придумать** как закодировать одно в другое

РАЗЛОЖЕНИЕ ЧИСЛА НА РАЗРЯДЫ

- * Число 42 можно представить как сумму $40 + 2$
- * Число 155 можно представить как сумму $100 + 50 + 5$
- * **Общая формула такая:** цифра * 10^{**} разряд + ...
- * Число 155 можно представить как сумму $1 * 10^{**} 2 + 5 * 10^{**} 1 + 5 * 10^{**} 0$
- * 10 тут является основанием системы счисления



ПРИМЕНИМ К БИНАРНОЙ СИСТЕМЕ

- * 0 – это 0
- * 1 – это 1
- * 2 – это 10, так как $1 * 2^{**} 1 + 0 * 2^{**} 0$
- * Вместо основания 10 используется 2
- * Такой алгоритм кодирования целых чисел в двоичный формат называется **прямым кодом**
- * Современные процессоры **нативно «понимают» такой формат**, но это не единственный способ кодирования

Число 1 0 1 1

1 0 1 1

Позиции (разряды)

Восьмёрки

Четвёрки

Двойки

Единицы

Степень двойки

$$2^3 = 8$$

$$2^2 = 4$$

$$2^1 = 2$$

$$2^0 = 1$$

Значение разряда

$$1 \times 8 = 8$$

$$0 \times 4 = 0$$

$$1 \times 2 = 2$$

$$1 \times 1 = 1$$

Итоговое число

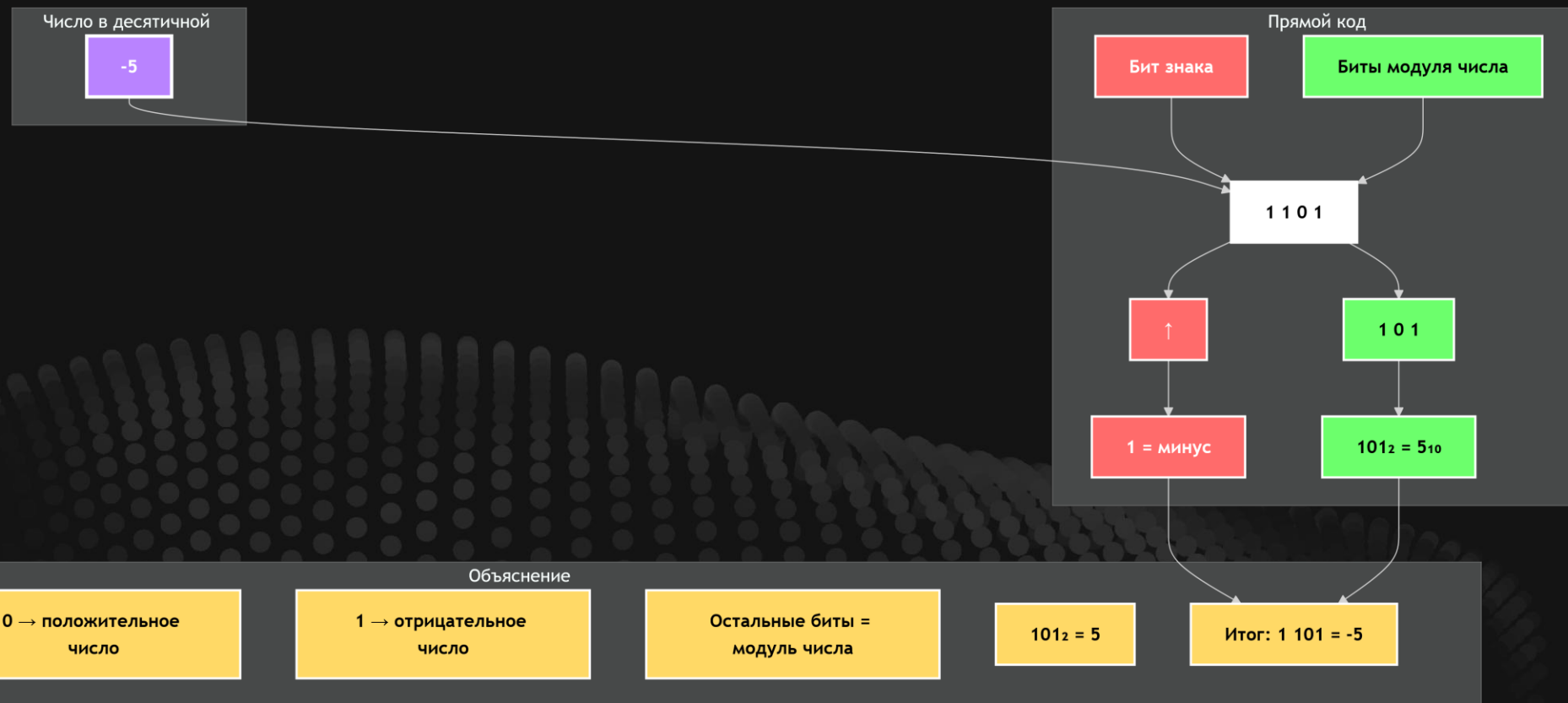
$$8 + 0 + 2 + 1 = 11 \text{ (в десятичной)}$$

РАЗМЕР ЧИСЛА

- * Для повышения эффективности операций с числами, **большинство числовых типов имеют явно ограниченный размер**
- * 1 байт, 2 байта, 4 байта, 8 байт и так далее
- * **Используя прямой код**, в одном байте можно закодировать 256 чисел
- * Например, от 0 до 255
- * Или от -128 до 127

ПРОБЛЕМА ОТРИЦАТЕЛЬНЫХ ЧИСЕЛ

- * В десятичной системе мы ставим перед числом знак -
- * В компьютере у нас только 0 или 1
- * Можно выделить крайне левый бит числа под знак
- * 0 за +, а 1 за -
- * Само число при этом кодируется прямым кодом

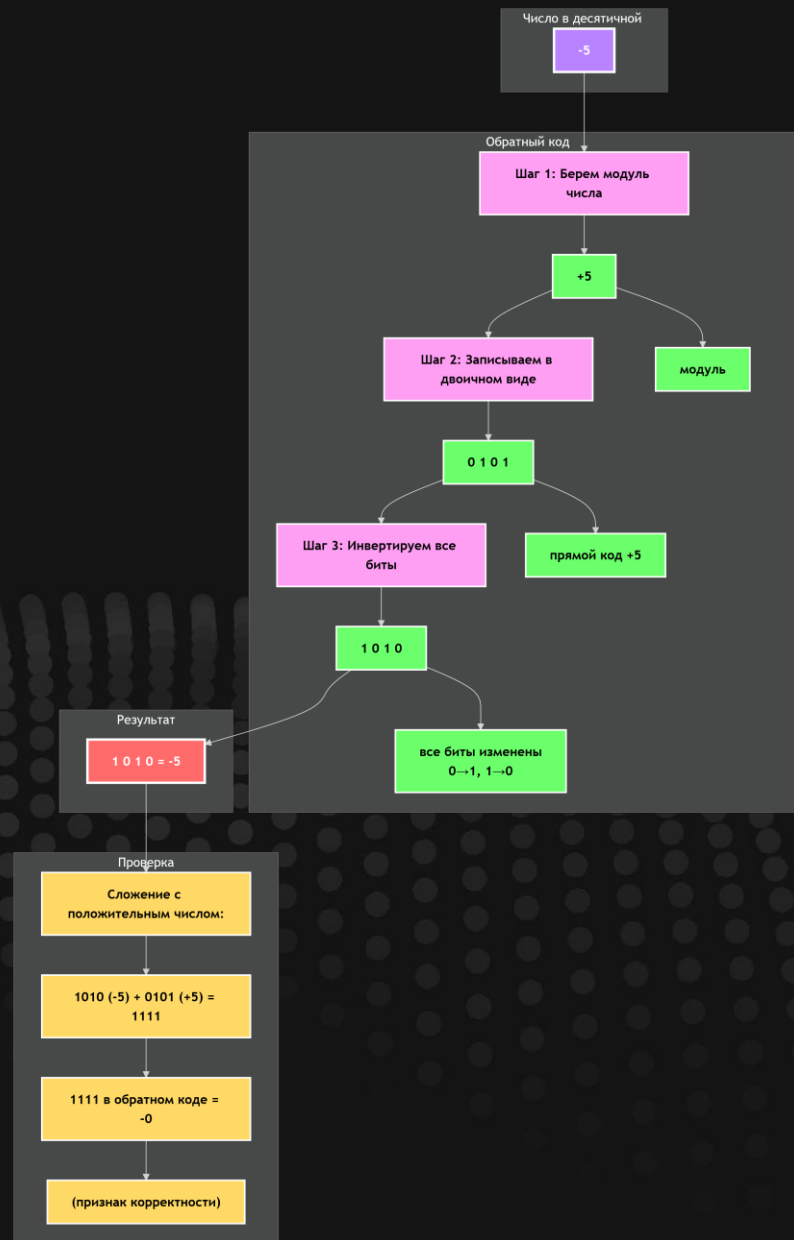


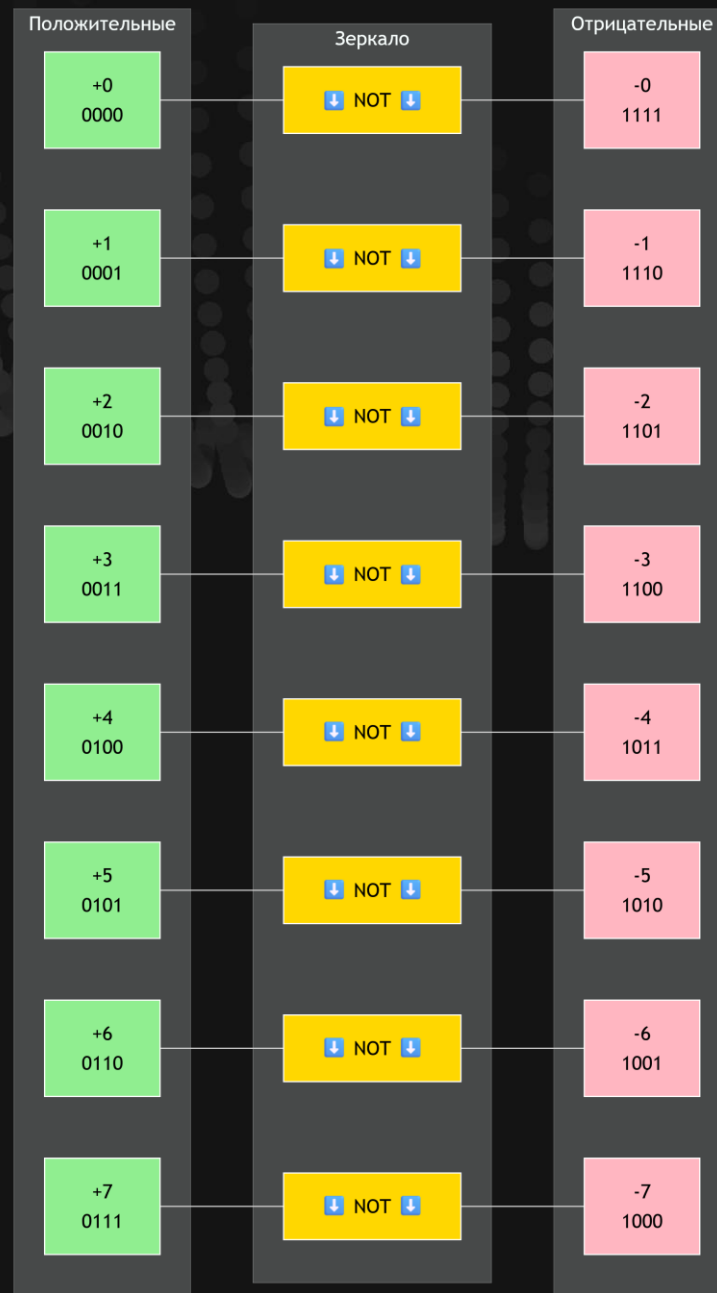
МИНУСЫ ТАКОГО ПОДХОДА

- * У нас появляется число -0 (1000)
- * Процессор должен поддерживать **отдельные операции для вычитания чисел**
- * А **площадь кристалла** не резиновая и не дешевая

ВЫЧИТАНИЕ ЧЕРЕЗ СЛОЖЕНИЕ

- * Существуют алгоритмы, которые позволяют закодировать число так, чтобы можно было делать вычитание используя сложение
- * Например, $7 - 9 \rightarrow 7 + (-9)$
- * Для инвертирование знака числа применяется операция побитового отрицания
- * Например, $7 - 9 \rightarrow 00111 + (-01001) \rightarrow 00111 + 10110$
- * Операция инвертирования бит очень проста для процессора
- * Такой алгоритм называется обратным кодом
- * Или дополнением до единицы



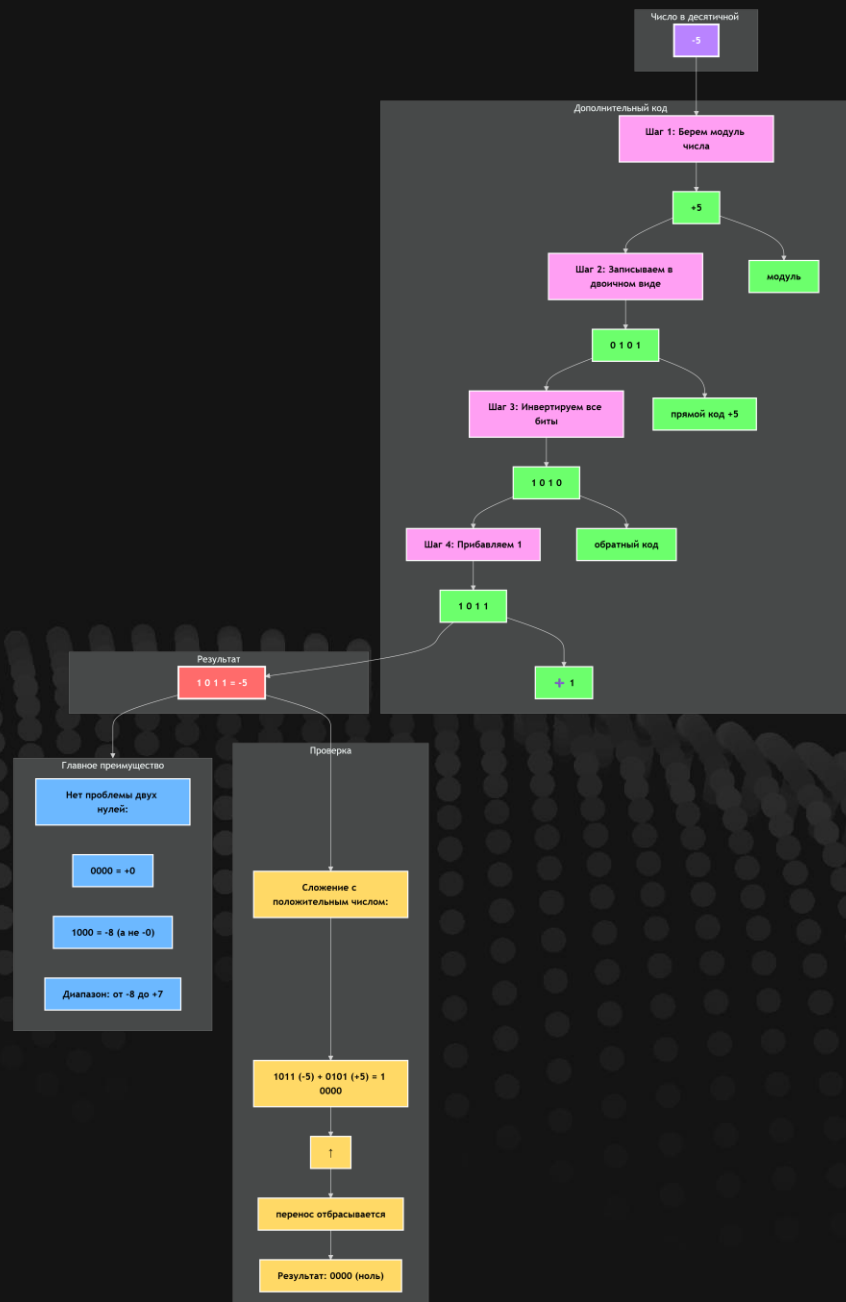


РЕФЛЕКСИРУЕМ

- * Теперь процессор может складывать и вычитать числа **используя одну и ту же операцию**
- * Главное **закодировать отрицательное число** через обратный код
- * А положительное кодируется через прямой код
- * Крайне левый бит **определяет знак**
- * Однако, у нас все еще есть число -0 и оно **добавляет проблем**

ДОПОЛНИТЕЛЬНЫЙ КОД

- * Модифицированный алгоритм для кодирования отрицательных чисел
- * Также известен как **дополнение до двух**
- * Позволяет избавиться от лишнего значения -0 и упростить арифметику
- * Логика такая же, как и обратного кода, но **после инвертирования бит делается +1**
- * Сейчас это **наиболее популярный алгоритм** для представления отрицательных целых чисел
- * Например, `console.log(-42 === ~42 + 1); // true`



СНОВА РЕФЛЕКСИРУЕМ

- * Алгоритмы прямого и дополнительного кода кодируют **только целые числа**
- * Для дробных чисел используются другие алгоритмы
- * Если наше число не может иметь знак, то **нам хватит просто прямого кода**
- * Для изменения знака числа нужно **инвертировать его биты и прибавить 1**
- * **0** существует в единственном экземпляре

А ТЕПЕРЬ СЛОЖИМ 10 И -10

$$\begin{array}{r} * \quad 01010 \\ + \quad 10110 \\ = \\ (1) 00000 \end{array}$$

- * 1 + 1 приводит к 0 и переносу 1 в следующий разряд
- * В результате получается число, **которое занимает больше бит**
- * Так как мы ограничиваем размер числа количеством бит, которое оно может занимать, то **все «лишние» биты отбрасываются**
- * Это называется **переполнением**

АЛГОРИТМ РАБОТАЕТ ИЗ-ЗА ПЕРЕПОЛНЕНИЯ?



В ТОЧКУ

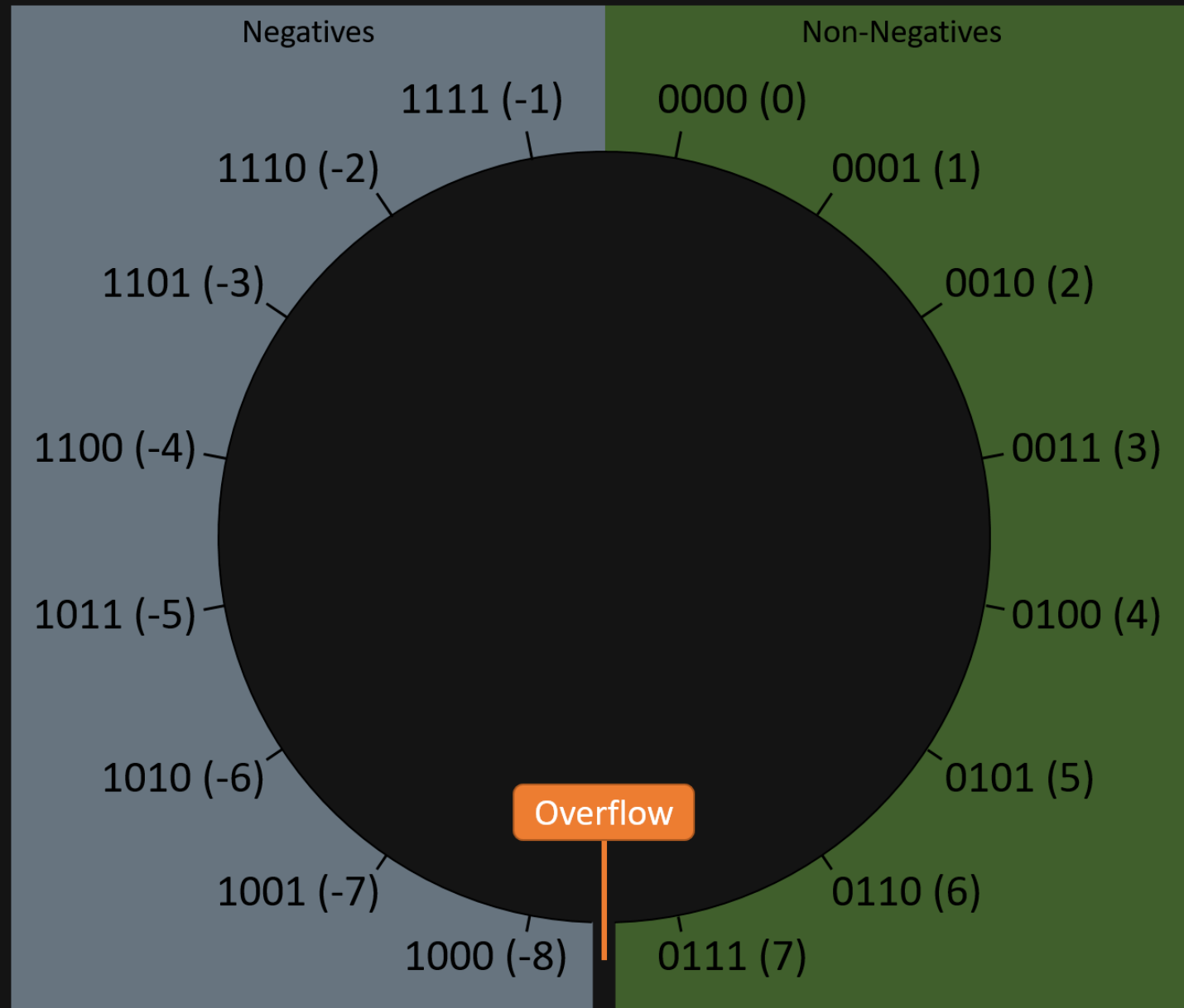
- * Эта еще одна причина, почему **числовые типы имеют фиксированный размер**
- * В 8-ми битах можно закодировать 256 значений ($2^{**} 8$)
- * В 16-ти уже 65 536 ($2^{**} 16$)
- * Биты не влазящие в диапазон **просто отбрасываются**
- * Благодаря этому мы **можем вычитать числа используя сложение**

НУЖНО «ПОНИМАТЬ» ПЕРЕПОЛНЕНИЕ

```
const a = new Uint8Array([255]);

a[0] += 2;

console.log(a[0]); // 1
                  // 1111 1111
                  // +
                  // 0000 0010
                  // =
                  // (1) 0000 0001
```



НАПРИМЕР

```
let a = 0;

// Из-за переполнения числа ходят по кругу:

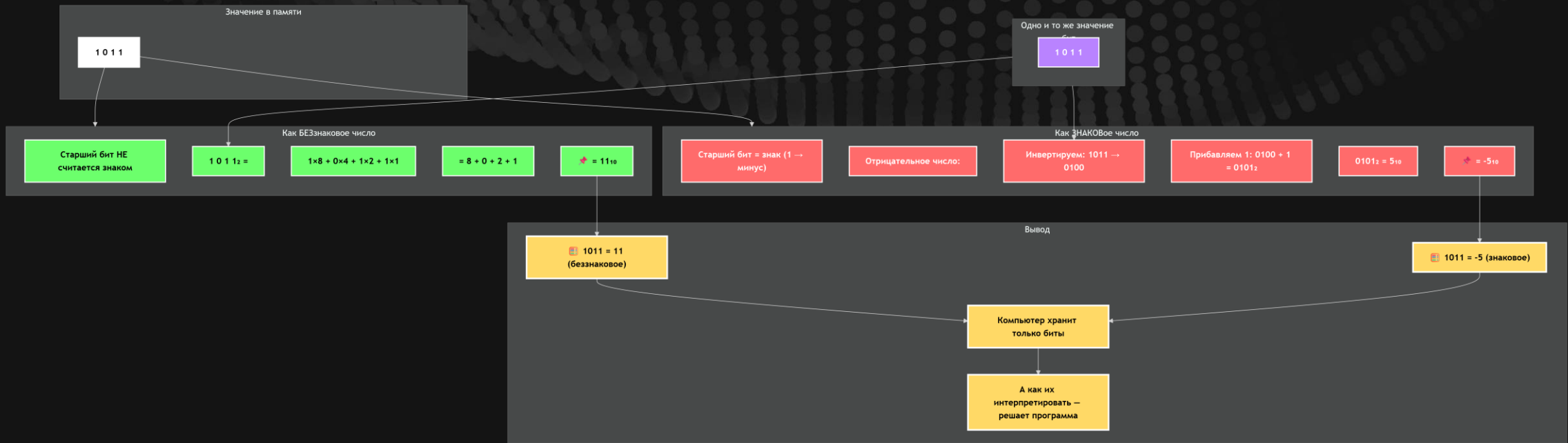
// 0 1 2 3 4 5 6 7
// 0 1 2 3 4 5 6 7
// ...
while (true) {
  console.log(a);

  // Для наглядности ограничиваю размер числа 3-мя битами
  a = (a + 1) & 0b111;
}
```

ВАЖНА ИНТЕРПРЕТАЦИЯ РЕЗУЛЬТАТА

- * Если наше число не может иметь знак, то в 8-ми битах мы можем хранить значение от 0 до 255 (256 значений)
- * А если может – от -128 до 127 (тоже 256 значений)
- * Мы можем посмотреть на одни и те же биты как на число со знаком или без

```
const a = new Int8Array([-125]);  
const b = new Uint8Array(a.buffer);  
  
console.log(a[0]); // -125  
console.log(b[0]); // 131
```



second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

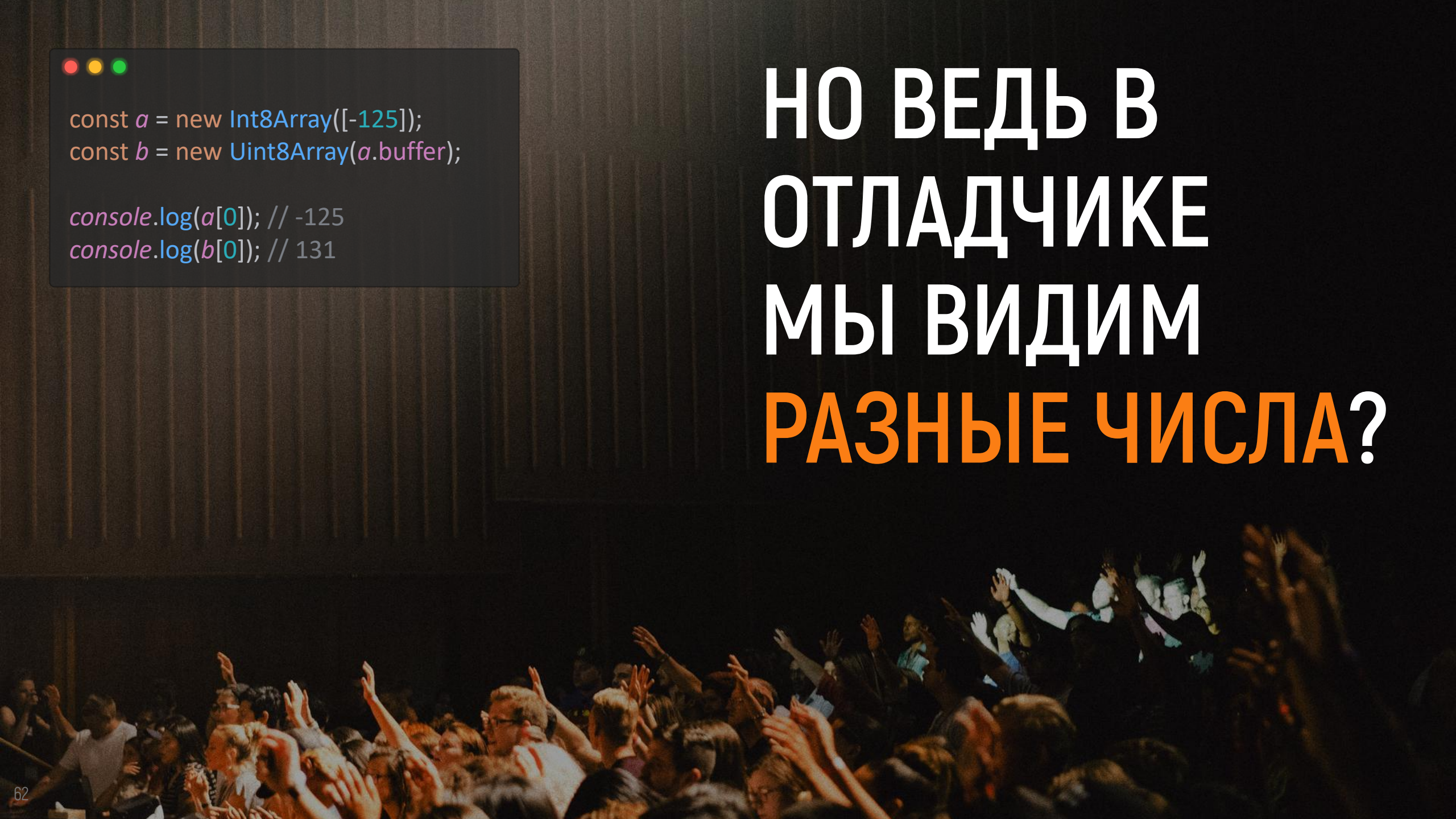
СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- * У нас будет отдельная лекция по кодированию чисел, а сегодня **главное понять смысл**
- * С практической точки зрения важнее всего понять **алгоритм прямого кодирования**
- * И механизм **переполнения числовых типов**

РЕФЛЕКСИРУЕМ

- * Процессор **не понимает того, что он вычисляет**
- * Например, у него есть операция «целочисленного сложения» и он **может применить ее к любым битам**
- * Как интерпретировать результат **решает программист**
- * Чтобы **наделить контекстом те или иные биты** в языках программирования используются типы



```
const a = new Int8Array([-125]);  
const b = new Uint8Array(a.buffer);
```

```
console.log(a[0]); // -125  
console.log(b[0]); // 131
```

НО ВЕДЬ В ОТЛАДЧИКЕ МЫ ВИДИМ РАЗНЫЕ ЧИСЛА?

ЭТО МОЖЕТ СБИВАТЬ С ТОЛКУ

- * Но при выводе на экран **«биты числа»** преобразуются в строку, которую мы уже и видим в отладчике
- * Каждый тип данных имеет **свой алгоритм кодирования для вывода на экран**
- * Одни и те же биты будут в итоге представляться так, **как это декларирует тип и программа его интерпретирующая**
- * Я могу прочитать JPG файл как изображение и вывести на экран
- * А могу прочитать его как одно большое число или последовательность чисел
- * Или вообще как что-то другое

МОЖНО ПОСМОТРЕТЬ НА СЫРЫЕ БИТЫ

```
> const a = new Float64Array([100.343])
< undefined
> a.buffer
< ▶ ArrayBuffer(8)
>
```

Консоль Инспектор памяти ×

ArrayBuffer(8) ×

0x00000000

00000000 CB A1 45 B6 F3 15 59 40
00000014
00000028
0000003C
00000050
00000064
00000078
0000008C
000000A0
000000B4
000000C8
000000DC

. E . . Y @

Little Endian ▾

Integer 8-bit	dec ▾	203 -53
Integer 16-bit	dec ▾	41419 -24117
Integer 32-bit	dec ▾	3058016715 -1236950581
Integer 64-bit	dec ▾	4636761427833889227
Float 32-bit	dec ▾	-0.00
Float 64-bit	dec ▾	100.34
Pointer 32-bit		0xB645A1CB ↗
Pointer 64-bit		0x405915F3B645A1CB ↗

ЛЮБОЙ ФАЙЛ МОЖНО ОТКРЫТЬ В СЫРОМ ВИДЕ

- * Для этого используются HEX редакторы
- * Такие программы **интерпретируют любой файл как поток чисел** в 16-тиричной или другой системе счисления
- * Ехе файлы, картинки, текст – все **можно открыть и отредактировать** таким образом
- * И, конечно же, можно все сломать 😊

ЧТО ЕЩЕ ЗА 16-ТИРИЧНАЯ СИСТЕМА СЧИСЛЕНИЯ?



В ЧЕМ СОЛЬ

- * Система счисления **определяет алфавит цифр** из которых строятся числа
- * В 10-тиричной системе счисления 10 цифр (от 0 до 9)
- * В 2-ичной системе 2 цифры (0 и 1)
- * В 16-тиричной системе 16 цифр (от 0 до 9, а потом от A до F)

ЧИСЛО В БОЛЕЕ БОГАТОЙ СИСТЕМЕ ВИЗУАЛЬНО ВЫГЛЯДИТ КОРОЧЕ



```
console.log((1255).toString(2)); // 10011100111  
console.log((1255).toString(8)); // 2347  
console.log((1255).toString(10)); // 1255  
console.log((1255).toString(16)); // 4e7  
console.log((1255).toString(36)); // yv
```

УДОБСТВО 16-ТРИЧНОЙ СИСТЕМЫ

- * Размер 16 кратен размеру байта из 8 бит, а значит любое число от 0 до 255 можно записать используя 2 буквы в такой системе
- * Сравните красный цвет в RGB (#FF0000) в разных системах счисления

```
console.log([0b1111_1111, 0b0000_0000, 0b0000_0000]); // [255, 0, 0]
console.log([0o377, 0o000, 0o000]); // [255, 0, 0]
console.log([0xFF, 0x00, 0x00]); // [255, 0, 0]
```

ПОЧЕМУ БЫ ВСЕГДА
НЕ ИСПОЛЬЗОВАТЬ
10-ТИЧНУЮ СИСТЕМУ?



ЭТО НЕУДОБНО

- * 10 не кратна 8, а значит мы не можем записать любое число от 0 до 255 одним количеством символов
- * Получается, что число **может быть одним символом, двумя или тремя**
- * Поэтому, при чтении сырых байт зачастую используется 16-тиричная система
- * Права файлов в Linux использую 8-миричную систему
- * Двоичная система удобная для создания битовых масок
- * Ну а для чисел-чисел используем привычную 10-тиричную

ОДИНАКОВОЕ С ВИЗУАЛЬНОЙ СТОРОНЫ ЧИСЛО БУДЕТ РАЗНЫМ



```
// Отладчик JS выводит числа в 10-тиричной системе  
console.log(parseInt("10", 2)); // 2  
console.log(parseInt("10", 8)); // 8  
console.log(parseInt("10", 16)); // 16  
console.log(parseInt("10", 36)); // 36
```

ВИЗУАЛЬНО РАЗНЫЕ ЧИСЛА МОГУТ БЫТЬ ОДИНАКОВЫМИ



```
console.log(0b1111_1111 === 255); // true  
console.log(0xFF === 255);         // true  
console.log(0xFF === 0o377);       // true
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ПОЙМИТЕ ГЛАВНОЕ

- * Система счисления позволяет **записать и прочитать число**
- * Число «двести пятьдесят пять» может быть записано как: 255, FF или 1111 1111
- * Но оно **все равно остается «двести пятьдесят пять»**

А ЧТО ТАМ БЫЛО ПРО ТИПЫ В ЯП?



НАМ НУЖНО ВЫДЕЛИТЬ КОНТРАКТЫ

- * Процессор **не понимает смысла** того, что он делает
- * У него есть набор поддерживаемых операций и программист должен сам решать, где, как и с чем их использовать
- * Чтобы облегчить себе жизнь, программисты придумали типы данных
- * Фактически, тип – это некоторое мнемоническое имя, которое **определяет множество данных закодированных определенным образом**
- * А также **набор контрактов и поддерживаемых операций**

НАПРИМЕР

- * Числа к числам можно применять арифметические операции
- * **Но результат этих операций зависит от конкретного типа числа**
- * Представление в памяти разных числовых типов тоже **может отличаться**
- * Строки, как и числа, можно складывать, но **алгоритм сложения будет абсолютно разный**
- * В конце концов, разные типы могут **по разному отображаться «на экране»**

ПРИВЕДЕНИЕ И ПРЕОБРАЗОВАНИЕ

- * Значения разных типов могут быть по-разному представлены в памяти
- * Процесс **конвертации одного типа в другой** называется преобразованием типа
- * Интерпретация представления данных одного типа в качестве значения другого типа (**без преобразования значения**) называется приведением типов



```
const a = new Float64Array([42.10]);

// ПРИВЕДЕНИЕ Float64Array к Uint8Array

const b = new Uint8Array(a.buffer);

// Uint8Array(8) [
//   205, 204, 204, 204,
//   204, 12, 69, 64
// ]
console.log(b);

// ПРЕОБРАЗОВАНИЕ Float64Array к Uint8Array

const c = new Uint8Array(a);

// Uint8Array(1) [ 42 ]
console.log(c);
```

ЯЗЫКИ ПРОГРАММИРОВАНИЯ И ТИПЫ

- * Большинство языков программирования предоставляют **набор встроенных стандартных типов** (числа, строки и так далее)
- * А также позволяют **создавать пользовательские типы** (структуры, классы)
- * Но есть языки и без поддержки встроенных типов
- * Например, языки ассемблера

РАЗНЫЕ ЯП ПО РАЗНОМУ ОБРАБАТЫВАЮТ ТИПЫ

- * Одни используют типы только на этапе компиляции
- * Другие реализуют их поддержку в рантайм
- * Есть и гибридные варианты
- * В случае с JS, то из-за его дизайна вся работа с типами происходит **исключительно в момент исполнения**
- * Но про это поговорим в следующей лекции

ТИПОВ МОЖЕТ БЫТЬ МНОГО

- * Просто потому, что **видов информации** также бывает много
- * А еще, есть типы, **которые состоят из значений других типов**
- * Такие типы называются **структурами данных**

СТРУКТУРА ДАННЫХ – ЭТО ТОЖЕ ТИП?



РАЗУМЕЕТСЯ

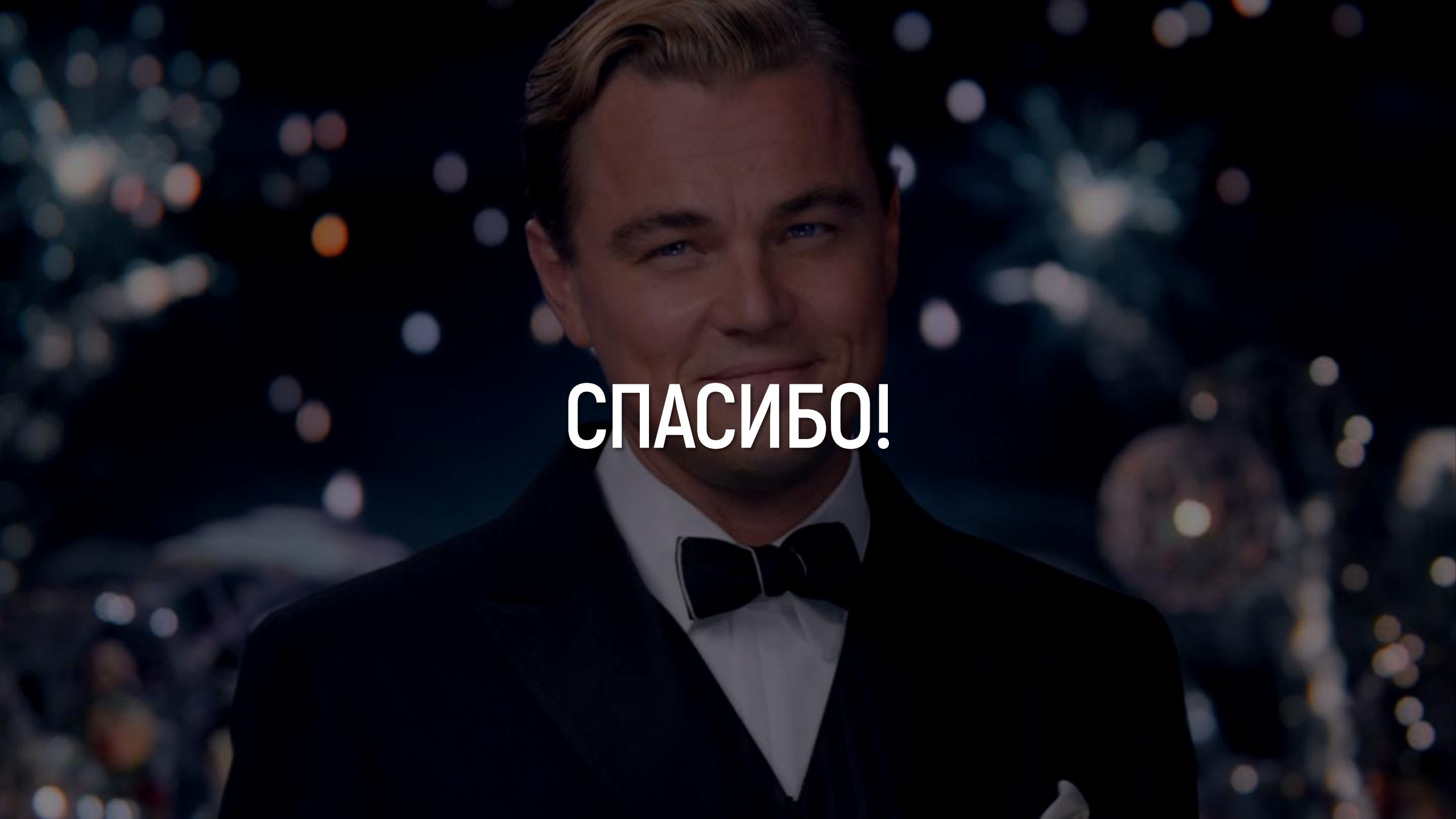
- ✱ Ведь **она тоже вводит контракты**: свойства и API
- ✱ И, даже как это должно быть представлено в памяти
- ✱ Например, массив или хеш-таблица

ДОБАВИМ АБСТРАКТНОСТИ

- * Тип может **просто описывать контракты на API**
- * При этом ничего не говоря, как именно они должны реализовываться
- * Такой тип называется **абстрактным**
- * Структуры данных **тоже могут быть абстрактными**
- * Например, стек или очередь

ПОДВЕДЕМ ИТОГИ

- * Программирование – **это вычисление над данными**
- * В конечном счете все сводится к тому «как представить данные в памяти» и «какие операции к ним применить»
- * **В силу физической архитектуры** современные компьютеры работают с двоичным представлением данных
- * Нам нужно **уметь кодировать и декодировать данные** в формат «понятный» вычислителю
- * Типы данных позволяют очертить **представление и контракты конкретных данных**

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with numerous out-of-focus, colorful bokeh lights in shades of blue, white, and orange, suggesting a night-time event or fireworks.

СПАСИБО!